

VAS Service Operations Control Tower

Sprint 2 — Database & Backend: Completion Instructions for Younes & Hiba

Sprint 1 (frontend prototype) is done. This document covers what each of us needs to finish Sprint 2: turning the hardcoded sample data into a real, persistent, shared database with a working API. Once both parts below are done, the dashboard will show real data that survives a page refresh.

PART 1 — HIBA (Backend & Database)

Goal: get a real Postgres database running and expose the endpoints the frontend needs.

1. Get a real database running

Sign up at **neon.tech** (free tier). Create a new project — this gives you a Postgres connection string in about 2 minutes.

Paste that connection string into backend/.env as:

```
DATABASE_URL=postgresql://...your-neon-connection-string...
```

Note: Make sure .env is listed in .gitignore before committing anything — never push the connection string to GitHub.

Then open Neon's built-in SQL editor (in their web dashboard), paste the full contents of database/schema.sql, and run it. That creates the users, projects, tasks, milestones, and risks tables and seeds them with the sample team/project data.

2. Add two small columns while you're already in the SQL editor

These aren't needed for Sprint 2 to demo, but they're needed for Sprint 4 (roles/permissions) right after, and it's much easier to add them now than to retrofit later. Run this after schema.sql:

```
ALTER TABLE users ADD COLUMN is_manager BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE projects ADD COLUMN lead_id UUID REFERENCES users(id);

-- optional but recommended: mark Nesrine (VAS Business Manager) as the manager
UPDATE users SET is_manager = TRUE WHERE name = 'Nesrine';
```

3. Add the 3 missing endpoints

Right now the backend only has “get tasks by one assignee” and “update task status.” The dashboard needs full lists of projects, users, and tasks. Add these:

backend/src/controllers/projects.controller.js

```
import { pool } from "../../config/db.js";

export async function getProjects(req, res) {
  const result = await pool.query("SELECT * FROM projects ORDER BY name");
  res.json(result.rows);
}
```

backend/src/controllers/users.controller.js

```
import { pool } from "../../config/db.js";

export async function getUsers(req, res) {
  const result = await pool.query("SELECT * FROM users ORDER BY name");
  res.json(result.rows);
}
```

Add to backend/src/controllers/tasks.controller.js

```
export async function getAllTasks(req, res) {
  const result = await pool.query(`
    SELECT id, title, project_id AS "projectId", assignee_id AS "assigneeId",
           status, due_date AS "dueDate", created_at AS "createdAt"
    FROM tasks ORDER BY due_date
  `);
  res.json(result.rows);
}
```

Note: The AS "projectId" aliases matter — Postgres returns snake_case column names by default (project_id), but the frontend expects camelCase (projectId). Without the alias, the data will load but the fields will silently be undefined on the frontend.

4. Wire up the routes

Create `projects.routes.js` and `users.routes.js` following whatever pattern `tasks.routes.js` already uses, plus one new line in `tasks.routes.js` for the `getAllTasks` route (e.g. GET /).

5. Mount the routes in server.js

```
app.use("/api/tasks", tasksRoutes);
app.use("/api/projects", projectsRoutes);
app.use("/api/users", usersRoutes);
```

6. Add CORS

The frontend (localhost:5173) and backend run on different ports — the browser will block requests unless the backend explicitly allows it:

```
npm install cors

// in server.js, near the top
import cors from "cors";
app.use(cors());
```

7. Test before handing off

Visit each endpoint directly in the browser and confirm real JSON comes back:

- `http://localhost:PORT/api/projects`
- `http://localhost:PORT/api/users`
- `http://localhost:PORT/api/tasks`

Push everything to GitHub, then message Younes with the port number the server runs on (e.g. 3000) — that's the one piece he can't guess and needs to wire up the frontend.

Hiba's checklist:

- Neon project created, DATABASE_URL set in backend/.env, .env confirmed in .gitignore
- schema.sql run successfully against Neon
- is_manager column added to users, lead_id column added to projects
- getProjects, getUsers, getAllTasks endpoints added and returning camelCase JSON
- Routes wired and mounted in server.js
- CORS installed and enabled
- All 3 endpoints tested directly in browser and return real data
- Pushed to GitHub, port number sent to Younes

PART 2 — YOUNES (Frontend)

Goal: swap the hardcoded sample data in App.jsx for real data pulled from Hiba's API.

1. Prep the fetch code now (don't wait idle)

Near the top of App.jsx, where PROJECTS, TASKS, and RESOURCES are hardcoded, add state and a fetch effect. Leave the hardcoded arrays in place until Hiba's endpoints are confirmed live — don't delete them yet.

```
const [projects, setProjects] = useState([]);
const [tasks, setTasks] = useState([]);
const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  Promise.all([
    fetch("http://localhost:PORT/api/projects").then(r => r.json()),
    fetch("http://localhost:PORT/api/tasks").then(r => r.json()),
    fetch("http://localhost:PORT/api/users").then(r => r.json()),
  ]).then(([projectsData, tasksData, usersData]) => {
    setProjects(projectsData);
    setTasks(tasksData);
    setUsers(usersData);
    setLoading(false);
  }).catch(err => {
    console.error("Failed to load data:", err);
    setLoading(false);
  });
}, []);
```

Note: Replace PORT with whatever number Hiba sends you once her server is running — don't guess.

2. Add a loading state

Wrap the dashboard so it doesn't flash empty before data arrives:

```
{loading ? <div>Loading...</div> : <YourExistingDashboardJSX />}
```

3. Update field names to match the real data shape

Once real data loads, tasks will have `assigneeId` and `projectId` instead of a plain `owner`: "Amir" string. Anywhere the code currently does something like:

```
t.owner === currentUser
```

it needs to become:

```
t.assigneeId === currentUser.id
```

Note: Don't make this change until Hiba's endpoints are live and you can see the real JSON shape — confirm field names match what's in her `getAllTasks` query before rewriting anything.

4. Stay out of backend files

Don't edit anything under `backend/` — if you want to help there too, coordinate with Hiba directly first so you're not both editing the same files independently and hitting merge conflicts.

Younes's checklist:

- Fetch code + loading state added to `App.jsx` (hardcoded arrays left in place as fallback for now)
- Port number received from Hiba, URLs updated
- Confirmed real data loads and matches expected shape (`projectId`, `assigneeId`, camelCase fields)
- `owner === name` comparisons updated to `assigneeId === id`
- Hardcoded `PROJECTS/TASKS/RESOURCES` arrays removed once real data confirmed working
- Dashboard tested end-to-end: refresh the page, data still there

Sprint 2 is done when: the dashboard loads real data from Postgres, a page refresh doesn't reset anything, and both of you have pushed to GitHub.